TECHNICAL GUIDE · DEPED BUKIDNON

GAMIFIED BULIG

BUKIDNON'S UNIFIED LITERACY AND INTERVENTION GATEWAY

How BULIG Works

The technology behind Bukidnon's Unified Literacy and Intervention Gateway, explained in plain language: the languages, the data, learning offline, the pronunciation check and every API used.

PHP 8 · MariaDB · SQL · HTML · CSS · JavaScript · Offline mode · Web Speech API · Phil-IRI scoring



1

THE BIG PICTURE

The three parts of BULIG

Every web system, BULIG included, is made of three parts that talk to each other. Think of a restaurant.

GEORGE SAYS
The device never talks to the database directly. It always asks the server first!



THE DINING TABLE

The device

The pupil's or teacher's phone, tablet or computer, using a web browser.

- Shows the pages
- Takes taps, typing, drawing, the microphone
- Keeps saved levels and waiting answers offline

⇄

THE KITCHEN · HOSTINGER

The server

A computer at Hostinger that runs BULIG's PHP program.

- Checks who is signed in
- Checks answers, gives stars and XP
- Builds every page

⇄

THE STOREROOM · MARIADB

The database

A MariaDB (MySQL) database with about 50 tables.

- Accounts, schools, sections
- Lessons, activities, answer keys
- Every answer, star, XP, badge

Why this matters

Because only the server reads and writes the database, a pupil can never see or change another pupil's data, and a teacher sees only their own pupils. The server checks the role before every page.

What is inside this guide

1 · The three parts	Device, server and database, and how they connect
2 · System architecture	The tiers, modules and services of BULIG
3 · Use case diagram	Who does what: pupils, teachers, administrators, BULIG
4 · The languages	PHP, SQL, HTML, CSS, JavaScript and the other tools
5 · One answer	What happens from the pupil's tap to the database
6 · Database schema	The entity-relationship diagram and all 51 tables
7 · Offline and sync	How lessons and answers are kept without internet
8 · Pronunciation check	How BULIG listens and scores reading (Phil-IRI)
9 · APIs used	Every browser and server API, and which work offline
10 · Keeping it safe	Passwords, tokens, private recordings
11 · Word list	The technical words, in plain language

2

HOW IT FITS TOGETHER

System architecture

A three-tier web application: the browser on the device, the PHP application on the server, and the MariaDB database. A service worker adds an offline layer.

GEORGE SAYS

Requests go down, answers come back up. The offline layer keeps learning going without internet!



USERS

3 roles, any device with a modern browser



Pupil
phone or tablet



Teacher
laptop or phone



Administrator
computer

↓ tap, type, speak ↑ pages, stars, sounds

CLIENT TIER · THE BROWSER

HTML, CSS, JavaScript

Presentation
HTML pages built by PHP; `theme.css` (Poppins, Liquid Glass, dark mode, phone layout)

Interaction
JavaScript: `learn.js`, `l1.js`, `l2a.js`, `coach.js`, `tour.js`, `app.js`

Device features
Web Speech (listen, read aloud), MediaRecorder, camera QR scan, Canvas drawing

Offline layer
Service worker `sw.js`, Cache Storage (saved levels), IndexedDB (waiting answers), `offline.js`

↓ HTTPS: GET pages · POST answers · JSON sync ↑ HTML · JSON results

APPLICATION TIER · HOSTINGER SERVER

Apache + PHP 8

Front controller · `public/index.php`
Every request enters here: it starts the session, sets the security headers (CSP), checks the role, sends the request to the right page or action, and checks the CSRF token on every form.

Accounts and sign-in
`actions.php`, `account_ids.php`, `teacher_accounts.php`, `saved_login.php`, `admin_pin.php`, `names.php`

Learning content
`coach.php`, `level1.php`, `level2a.php`, `level2_cards.php`, `level4.php`, `story.php`, `reading_check.php`

Checking and rewards
`scoring.php` (answer keys, stars), `level4.php` (Phil-IRI miscues), `post_gate.php`, `reader_level.php`, `growth.php`

Teaching tools
`staff_ui.php`, `sections.php`, `teacher_demo.php` (Class Demo), `cards.php`, `section_report.php`, `insights.php`

Administration
`admin.php`, `schools.php`, `school_page.php`, `directory.php`, `checkup.php`, `ratings.php`

Offline, alerts, imports
`offline.php` (manifest, sync), `reach.php` (Web Push), `notify.php`, `pupil_import.php`, `xls_reader.php`

↓ SQL through PDO (prepared statements) ↑ rows

DATA TIER

outside the public folder

MariaDB / MySQL
51 tables: accounts, schools, lessons, answers, progress, rewards, logs (chapter 6)

File storage
`storage/recordings` (voice, private), module pages and PDFs, backups

Uploads
`public/assets/uploads`: profile photos and school logos

External · speech-to-text service
Used by the browser's SpeechRecognition (Google for Chrome, Apple for Safari). No key, no payment.

External · push service
Delivers reminder notifications (Google, Apple, Mozilla), signed by BULIG with VAPID keys.

3

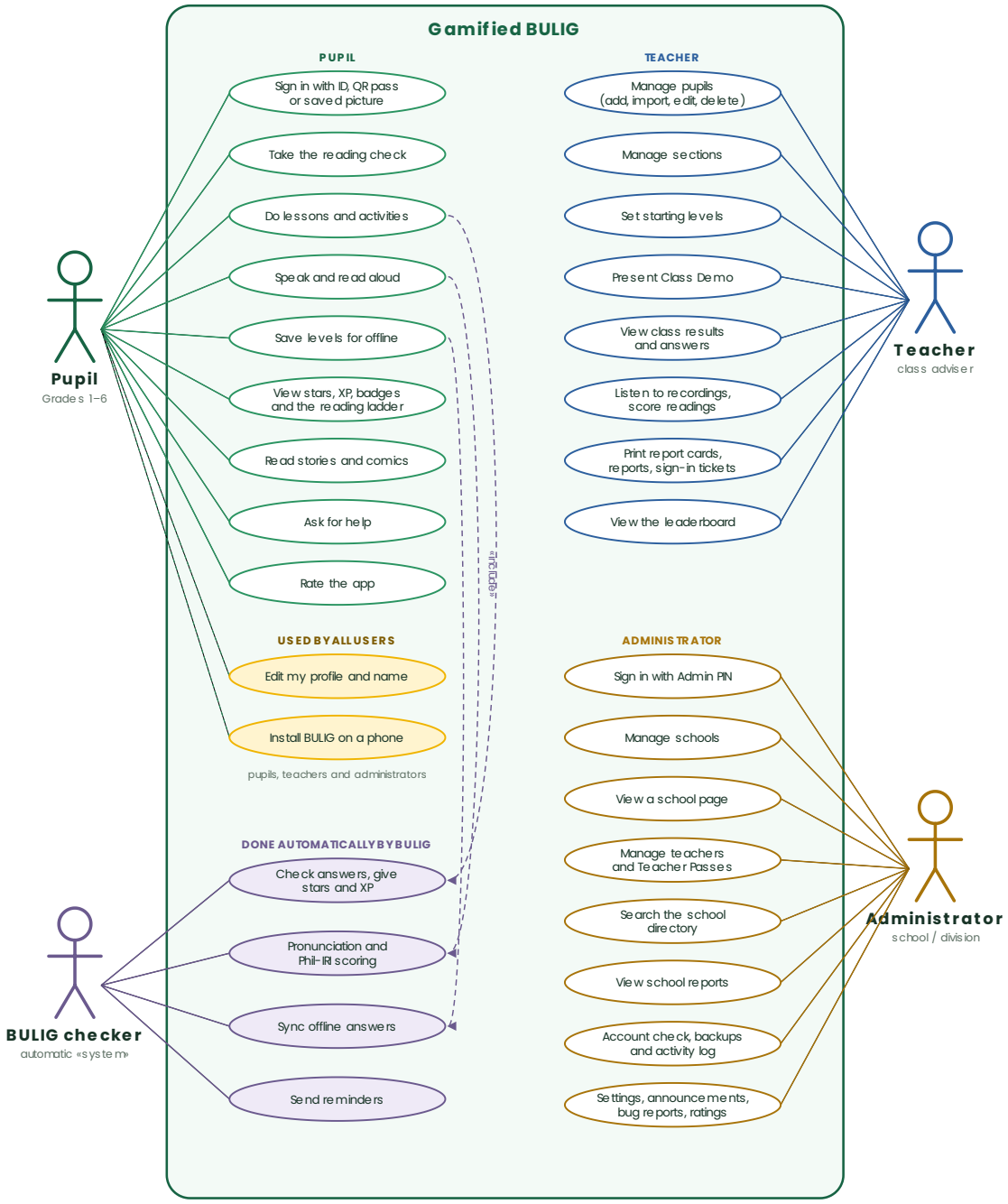
WHO DOES WHAT

Use case diagram

The actors (people and the automatic checker) and what each can do in BULIG. Lines show who uses each feature; dashed arrows marked «include» are steps BULIG always does as part of another.

SOFIE SAYS
Pupils learn, teachers guide, administrators manage, and BULIG checks everything automatically!





4

BUILT WITH

The languages and tools

BULIG uses no big ready-made framework. It is written directly in the standard web languages, which keeps it light for slow school internet and simple hosting.

JARVIS SAYS
Only the public folder is open to the internet. The program and passwords stay hidden!



LANGUAGE OR TOOL	RUNS ON	WHAT IT DOES IN BULIG
PHP 8 public/index.php, app/*.php	server	The brain. Signs people in, builds every page, checks answers, counts stars and XP, makes report cards, IDs and Teacher Passes.
SQL · MariaDB / MySQL database/schema.sql, migrations/	database	The language for saving, finding, changing and deleting data. Updates with new tables come as .sql files imported in phpMyAdmin.
HTML written by PHP	device	The structure of each page: headings, buttons, forms and pictures.
CSS public/assets/theme.css	device	The look: colours, Poppins font, Liquid Glass, dark mode and the phone layout.
JavaScript public/assets/*.js	device	Everything that moves without loading a new page: tapping answers, drawing, the microphone, tours, sounds, offline saving and syncing.
Service worker public/sw.js	device	A background helper that opens BULIG's saved pages when there is no internet.
Web Speech API coach.js, l1.js, rcheck.js	device	Turns the pupil's voice into text (listening) and reads text aloud (the Listen button).
Python tests/, tools/	developer	Not part of the live site. Runs the 284 automatic tests and updates the file fingerprints in bulig-check.php.
Apache + .htaccess public/.htaccess	server	The web server at Hostinger. .htaccess sets the public folder and how long phones may keep pictures.

Why the update ZIPs never include config/database.php

Only `public/` can be opened from the internet. `app/`, `config/` and `storage/` (voice recordings) sit outside it. `config/database.php` holds your Hostinger database password, so it is never replaced by an update.

5

STEP BY STEP

One answer, from tap to database

What happens when a pupil with internet answers a card and taps Submit.

BRUNO SAYS
Every answer is checked by the server, so stars and XP are always fair.



1 The browser sends the answer

JavaScript packs the answer into a form and sends it to the server (an HTTP `POST` to `index.php`) together with a secret form token and the sign-in cookie.

2 PHP checks who is asking

The server reads the session cookie to know which pupil this is, and checks the form token, so another website cannot send answers in the pupil's name.

3 PHP checks the answer

It looks up the answer key for that card and compares. Each wrong try takes away a third of the card's points. 80% or more earns at least 2 stars, which unlocks the next activity.

4 SQL saves it

PHP runs SQL such as `INSERT INTO activity_completion …`, adds XP in `pupil_xp`, marks the day in `learning_days` and checks badges.

5 The page answers back

The server replies with the result. JavaScript shows the stars, plays the sound and shows the character. Teacher pages read the same tables, so the teacher sees it right away.

The main tables (a few of about 50)

users	Every account: pupil, teacher, admin, with ID, name, school and hashed password
schools · sections · teacher_pupils	Which school each person belongs to, the sections, and which teacher has which pupil
bulig_levels · modules · lessons · activities	The learning content: Levels 1 to 7, lessons and every activity card
activity_completion	Every answer given, its score, and BULIG's check
pupil_progress · pupil_xp · learning_days	Lessons finished, XP earned, and the days a pupil learned
audit_log	The Activity log: sign-ins, deletions, name changes

6

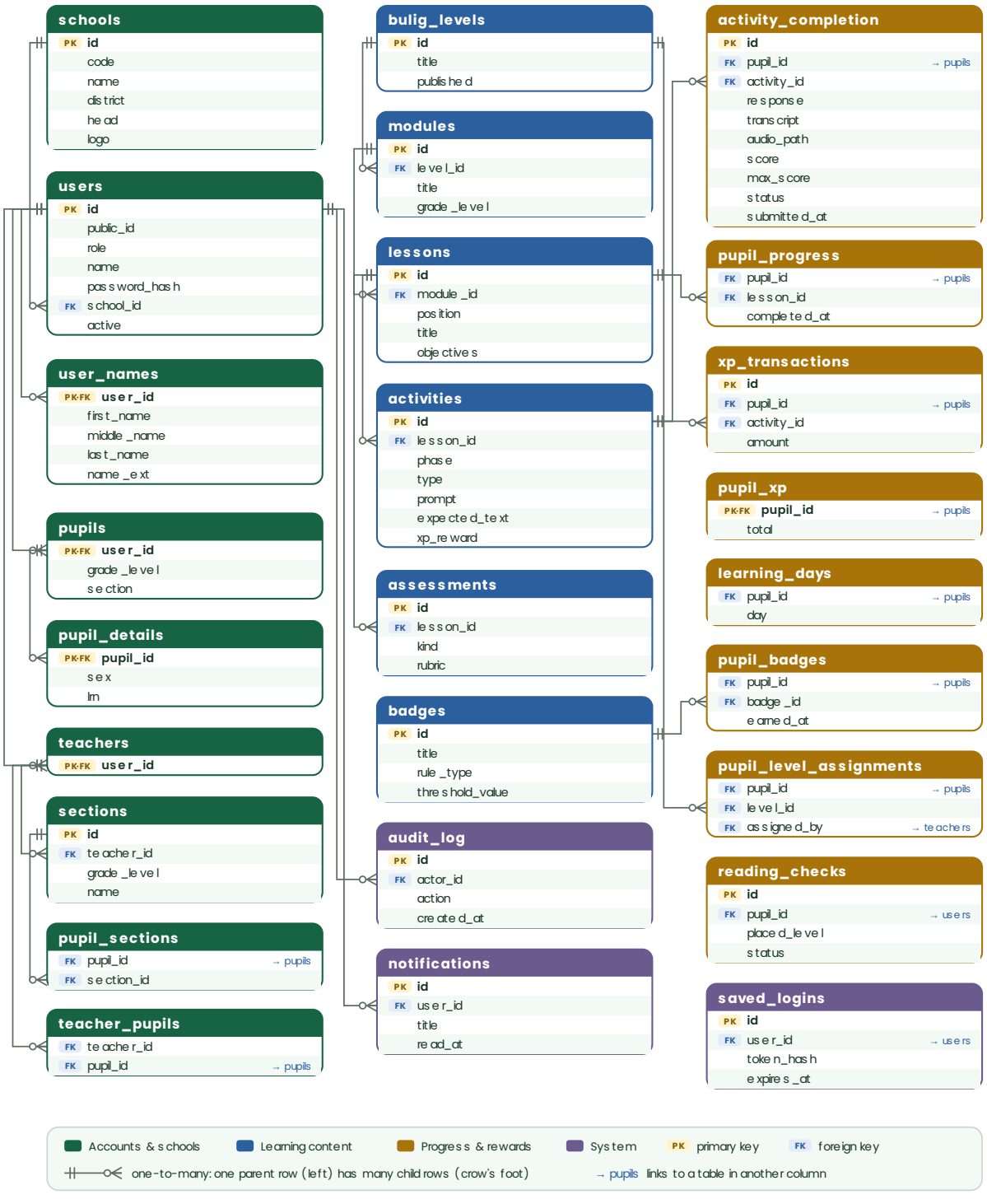
HOW DATA IS ORGANISED

Database schema

The main tables of the MariaDB database and how they link. Each box is a table; PK is its primary key and FK points to a row in another table.

BOB SAYS
Every answer links to one pupil and one activity, so the reports always add up!





Entity-relationship diagram of the 27 core tables (of 51). The full list follows on the next page.

6

DATABASE, CONTINUED

All 51 tables

Every table in the database, grouped by what it keeps.

Accounts and schools

`users` every account: ID, role, name, hashed password, school

`schools` code, name, School ID, district, head, logo

`pupils` · `teachers` · `admins` details for each role

`user_names` first, middle and last name, extension

`pupil_details` sex, LRN

`sections` · `pupil_sections` · `teacher_pupils` classes and who teaches whom

`grade_levels` Grades 1 to 6

`pupil_cards` sign-in ticket keys (QR)

`saved_logins` "Remember me" keys

`admin_security` Admin PIN

`id_sequences` · `account_id_history` next ID numbers, old IDs

`login_attempts` wrong-password limits

Learning content

`bulig_levels` Levels 1 to 7

`modules` a level's module for a grade

`lessons` lessons in a module

`activities` every activity card, prompt and expected text

`assessments` · `assessment_questions` · `questions` · `answers` pre-tests and post-tests

`source_pages` pages of the official module

`content_issues` reported content problems

Answers and progress

`activity_completion` every answer, recording, transcript and score

`activity_drafts` unfinished answers

`response_history` earlier versions of an answer

`assessment_attempts` test results

`reading_assessments` reading word-match results

`reading_checks` first reading check and placement

`pupil_level_assignments` starting level

`pupil_progress` lessons finished

`learning_days` · `pupil_streaks` days learned, streaks

`help_requests` "Ask for help"

Rewards

`pupil_xp` · `xp_transactions` XP total and each award

`badges` · `pupil_badges` badges and who earned them

`rewards` · `pupil_rewards` XP rewards

System

`audit_log` the Activity log

`notifications` the bell

`push_subscriptions` · `push_queue` phone reminders

`app_ratings` pupil and teacher ratings

`settings` school info, announcements, preferences

`schema_migrations` which .sql updates are imported

7

NO INTERNET? NO PROBLEM

Learning offline, then syncing

Without internet the device cannot reach the server. So BULIG keeps the lessons on the device beforehand, and keeps the pupil's answers on the device until the internet comes back.

TOBY SAYS
Save a level while you have internet. Then you can learn anywhere!



Box 1 · Cache Storage

Holds the saved lessons. "Save for offline" downloads every page and picture of a level, one box per pupil per level.

```
bulig-off-<pupil>-L<level>  lesson pages
+ pictures
bulig-login                 the sign-in
page
bulig-who                   pupil using
this device
```

Box 2 · IndexedDB

A small database inside the browser named `bulig-offline`. Each answer given offline waits here with its real time.

```
{ uid: 12,        // the pupil
  aid: 345,       // the activity
  at: "2026-10-28T09:14:03Z",
  fields: [ the answer … ] }
```

While offline

1. **The saved page opens**
   The service worker (sw.js) sees there is no internet and opens the copy from Cache Storage, only for the pupil who saved it.
2. **The answer waits**
   On Submit, offline.js puts the answer in IndexedDB, marks the activity done on the device and moves on. The page shows "answers waiting".

When the internet is back

1. **BULIG notices**
   The browser's "online" event fires; BULIG also checks every 60 seconds.
2. **It asks who is signed in**
   If signed out, it signs in again with the "Remember me" key.
3. **It sends each answer, oldest first**
   with `offline_at`, the real time it was given.
4. **The server checks the time**
   Not in the future, not older than 60 days, then saves it.
5. **The device clears its copy**
   only after the server confirms. A dropped connection just waits for the next try.

Good to know

- A level must be saved while there is internet. Waiting answers live only on that one device: clearing browser data deletes them before they are sent.
- Card-by-card checking needs the server. Offline, BULIG says "Saved. BULIG will check it when you finish" and checks them on sync.
- The pronunciation check needs internet (see chapter 8). Offline, the voice recording is kept and the teacher can listen to it.

8

LISTENING TO READERS

How the pronunciation check works

BULIG works in two steps: the browser turns the voice into words, then PHP compares those words with the words the pupil should have said.

MAX SAYS
Read clearly into the microphone, and BULIG will tell you how you did!



Step 1 · The browser listens

BULIG uses the Web Speech API's `SpeechRecognition`, set to English (`en-US`), asking for up to 5 guesses. Chrome uses Google's speech service, so listening needs internet. On iPhone, BULIG only listens and falls back to recording if listening fails.

The heard words are added to the answer, e.g. `said: the cat set on mat`.

Step 2 · PHP compares the words

Both texts are cleaned (small letters, no punctuation). Then:

- **Say the word (Level 1):** the share of expected words heard.
- **Read a passage (Level 4, Phil-IRI):** the passage and the heard words are lined up with the fewest changes (edit distance), and each change is named as a miscue.

MISCUE	HOW BULIG RECOGNISES IT	EXAMPLE
Mispronunciation	A near miss: same sound key (`metaphone`), or few letters different (`levenshtein`, up to a third of the word), or same first 3 letters of a long word	sat → set
Substitution	A different word in that place	house → home
Omission	A passage word that was not heard	the big dog → the dog
Insertion	An extra word not in the passage	the dog → the little dog
Repetition · Transposition · Reversal	Words said again · two words swapped · a word said backwards	the the · red big · was → saw

Worked example

Passage: "The cat sat on the big red mat" · **Heard:** "the cat set on the red mat"

the	cat	sat	on	the	big	red	mat
the	cat	set	on	the	not heard	red	mat
RIGHT	RIGHT	MISPRONOUNCED	RIGHT	RIGHT	OMISSION	RIGHT	RIGHT

75% Accuracy = (8 words - 2 miscues) ÷ 8 words × 100 → Frustration level

98% and up	90% – 97.9%	Below 90%
Independent	Instructional	Frustration

9

FOR THE PANEL

The APIs BULIG uses

An API (Application Programming Interface) is a ready-made set of functions a program calls to use a feature it did not build itself, like the microphone, the camera or the database.

JARVIS SAYS
No paid services, no API keys. Only standard browser and PHP features!



The short answer

BULIG uses **no paid or third-party APIs and no API keys**. It is built on the standard **Web APIs** every modern browser provides, the **built-in functions of PHP** on the server, and two small open-source libraries stored inside BULIG.

16 Web APIs in the browser

6 PHP functions and extensions

2 open-source libraries, stored locally

0 paid services or API keys

A. Web APIs (built into the browser)

API	WHAT BULIG USES IT FOR	OFFLINE?
SpeechRecognition Web Speech API	Hears the pupil speak or read aloud: pronunciation check (Level 1), Phil-IRI reading score (Level 4), reading check.	**Needs internet**
SpeechSynthesis Web Speech API	The Listen button, the characters' voices, word-by-word story read-along.	**Works**
MediaRecorder	Records the pupil's voice so the teacher can play it back.	**Works, sent later**
getUserMedia	Opens the microphone for recordings and the camera for QR sign-in.	**Works**
Barcode Detection API	Reads the QR code on the Reading Pass, Teacher Pass and Admin Pass.	**Works**
Service Worker API	Opens saved lessons offline, shows reminder notifications, makes BULIG installable.	**Works**
Cache Storage API	Keeps downloaded levels (pages, pictures) and the sign-in page on the device.	**Works**
IndexedDB	Keeps answers given offline, with their real time, until they are synced.	**Works**

API	WHAT BULIG USES IT FOR	OFFLINE?
Web Storage localStorage, sessionStorage	"Remember me" accounts, dark mode, chosen voice, tours already seen.	**Works**
Fetch API + FormData	Card checks, sending answers, the offline sync, loading lists without reloading.	**Waits for internet**
Online / Offline events	Starts the sync the moment the internet returns; shows "answers waiting".	**Works**
Push API + Notifications	Reminders on teachers' and administrators' phones.	**Needs internet**
Canvas API	Drawing pad, QR codes on passes, saving the Top 10 leaderboard as a picture.	**Works**
Web Audio API	Sound effects: correct, try again, clicks, celebrations.	**Works**
Fullscreen · Vibration · Clipboard	Full screen Class Demo and stories; a buzz on a QR scan; copying a password.	**Works**
Web App Manifest	"Install BULIG on this phone", so it opens like an app.	**Works**

9

APIS, CONTINUED

Server APIs and libraries

On the server, BULIG uses functions that come built into PHP. Two small libraries are stored inside BULIG, so they need no internet.

BOB SAYS
Every block of BULIG is built from standard parts. Nothing to pay, nothing to renew!



B. Server APIs (PHP, built in)

FUNCTION OR EXTENSION	WHAT BULIG USES IT FOR
PDO (MySQL / MariaDB)	Every read and write to the database, with prepared statements, so typed text can never change the SQL (protection against SQL injection).
password_hash · password_verify	Stores passwords as a scrambled code (bcrypt) and checks them at sign-in.
random_bytes	Secure random values: form tokens, saved-account keys, starter passwords, upload file names.
metaphone · levenshtein	The pronunciation check: a "sound key" so words that sound alike match, and a count of how many letters differ, to spot near misses.
OpenSSL	Signs and encrypts reminder notifications (the Web Push standard with VAPID keys), written in BULIG's own code.
ZipArchive · finfo	Reads uploaded Excel (.xlsx) class lists; checks uploaded pictures are really PNG, JPEG or WebP.

C. Open-source libraries (stored inside BULIG)

LIBRARY	WHAT BULIG USES IT FOR
qrcode.js	Draws the QR codes on Reading Passes, sign-in tickets and Teacher Passes.
html2canvas	Turns the Top 10 leaderboard into a picture the teacher can save or print.

The only outside services

Two come automatically with the device and need no key or payment: the browser maker's **speech-to-text service** (used by SpeechRecognition) and the phone's **push service** (Google, Apple or Mozilla) that delivers reminders. BULIG never sends pupil names or answers to them; the speech service only hears the audio while the pupil speaks.

Offline at a glance

Lessons, pictures, stories, instructions	Work offline (after saving the level)
Answering, matching, drawing, Listen button	Work offline
Checking each card, stars and XP	Done when the answers sync
Voice recording	Kept, sent later
Automatic pronunciation scoring	Needs internet; offline the teacher scores the recording

10

PROTECTION

How BULIG keeps data safe

Pupils' names, answers and voices are personal data. These are the locks BULIG uses.

CLAUDE SAYS
Passwords are scrambled and recordings stay private. Only the right people can see them!



Passwords never stored as typed

password_hash turns each password into a scrambled code. Even someone who sees the database cannot read them.

A secret token on every form

The CSRF token stops another website from making a signed-in teacher delete or change things.

A strict security policy

The Content-Security-Policy lets the browser run only BULIG's own scripts.

Private voice recordings

Kept in storage/recordings, outside the public folder. Only the pupil and their teacher can play them.

Each role sees only its part

Before every page PHP checks the role: pupils cannot open teacher pages; teachers see only their pupils.

An Activity log

Records sign-ins, failed passwords, deleted accounts and name changes, so the administrator can check what happened.

11

PLAIN LANGUAGE

Word list

The technical words in this guide, in plain language.

SOFIE SAYS
Now you know how BULIG works. You can explain it to anyone!



Server	The computer at Hostinger that runs BULIG for everyone.
Database · table	The organised storeroom of all data; a table is like one spreadsheet in it.
Query	One SQL instruction, like "find this pupil's stars".
Session · cookie	How the server remembers who is signed in between pages.
Request (GET · POST)	A message from browser to server: GET asks for a page, POST sends something.
Migration (.sql file)	A file that adds new tables or columns during an update.
Cache · IndexedDB	Saved copies of pages on the device · a small database inside the browser.
Service worker · sync	A background script that answers when offline · sending waiting answers once online.
Speech recognition	Turning spoken words into written text.
Edit distance · metaphone	Fewest changes to turn one text into another · a sound key for an English word.
Fingerprint	A short code from a file; bulig-check.php uses it to find missing or old uploads.